

제 5장 비트맵과 애니메이션

2026년 1학기 윈도우 프로그래밍

학습 목표

- 학습 목표

- 비트맵 형식의 그림 파일을 불러 화면에 출력할 수 있다.
- 더블 버퍼링 기법을 이용해 비트맵 그림 파일로 애니메이션을 만들 수 있다

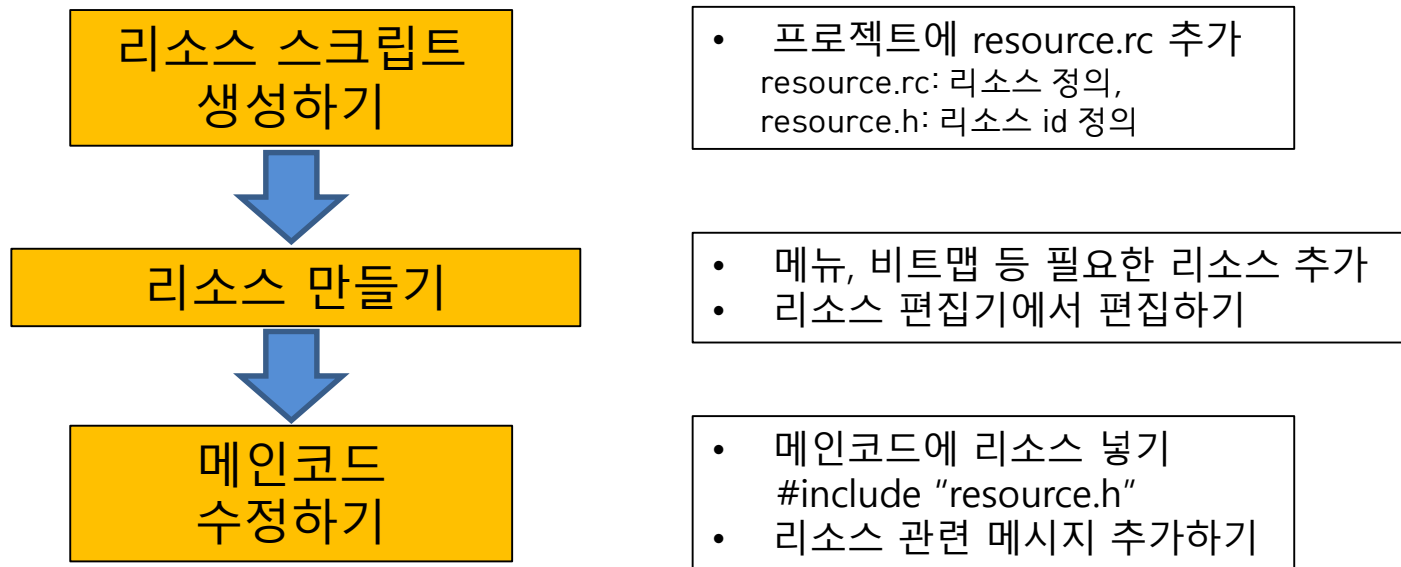
- 내용

- 비트맵
- 애니메이션 만들기
- 더블 버퍼링 사용하기
- 마스크 사용하기

1. 리소스 사용하기

- 리소스 (Resource)
 - 메뉴, 아이콘, 커서, 다이얼로그, 액셀러레이터, 비트맵 등 사용자 인터페이스를 구성하는 자원들로 읽기 전용 정적 데이터를 말한다.
 - 리소스는 프로그램 실행 중 변경되지 않는다.
 - C/C++과 같은 언어로 관리하지 않고 리소스 스크립트(Resource Script; .rc)파일로 관리한다.
 - 윈도우 프로그래밍에서는 리소스로 소스코드와 별도로 만들고 컴파일하며, 링크 시 최종 실행파일에 합쳐진다.

• 리소스 사용하기



2. 비트맵

- 컴퓨터 이미지(image)

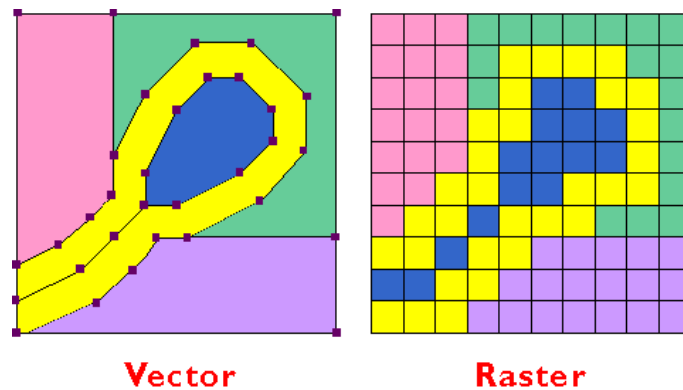
- 전자적인 형태로 만들어지거나 복사되고, 저장된 그림이다.
- 2차원 이미지 종류: 래스터 이미지(raster images), 벡터 이미지(vector images)

- 래스터 이미지 (raster images)

- 비트맵 이미지로 그래픽 객체들을 구성하고 있는 픽셀 (PIXEL, Picture Element) 값들을 그대로 저장하는 방식
- 사진처럼 디테일한 표현이 가능하다.
- 그림의 복잡도에 관계없이 그림 파일의 크기는 일정하지만 크기가 변경되면 화질이 저하된다.
- 대표적인 포맷: JPEG, PNG, GIF, BMP 등

- 벡터 이미지 (vector images):

- 이미지를 표현하기 위하여 수학 방정식을 기반으로 점, 직선, 곡선, 다각형과 같은 물체를 사용하는 방식
- 무한 확대, 축소가 가능하고 파일용량이 작다. 변형에 용이하다.
- 사진과 같은 사실적 표현에는 한계가 있다.
- 대표적인 포맷: SVG, AI, EPS, PDF 등



• 이미지 종류

포맷	특징	장점	단점	활용 예시
BMP (Bitmap)	마이크로소프트가 만든 기본 포맷, 압축 없음	단순 구조, 윈도우 기본 지원	파일 크기가 매우 큼	윈도우 내부 그래픽 처리, 간단한 이미지 저장
JPEG (JPG)	손실 압축 방식	파일 크기 작음, 사진에 적합	반복 저장 시 화질 저하, 투명도 미지원	디지털 사진, 웹 이미지
PNG	무손실 압축, 알파 채널(투명도) 지원	선명한 품질 유지, 투명 배경 가능	파일 크기가 큼	아이콘, 로고, UI 그래픽
GIF	256색 제한, 간단한 애니메이션 지원	움직이는 이미지 제작 가능, 용량 작음	색상 제한, 사진에는 부적합	애니메이션 아이콘, 간단한 웹 그래픽
TIFF	무손실 압축, 고해상도 지원	인쇄·출판용 고품질	파일 크기 큼	스캐닝, 출판, 전문 인쇄
WEBP	구글 개발, 손실/무손실 모두 지원	JPEG보다 작은 용량, PNG처럼 투명도 지원	일부 프로그램 호환성 부족	웹 최적화 이미지

- 윈도우 OS에서 지원하는 비트맵은 두가지이다.
 - 장치 종속 비트맵: DDB(Device Dependent Bitmap)
 - 장치 독립 비트맵: DIB(Device Independent Bitmap)

종류	설명	특징	활용
DDB (Device Dependent Bitmap)	장치 종속 비트맵	- 출력 장치(모니터, 프린터 등)에 맞춰 저장됨 - 디스플레이 컨텍스트(DC)에 의존 - 다른 장치에서 동일하게 표시되지 않을 수 있음	GDI에서 화면 출력 시 사용
DIB (Device Independent Bitmap)	장치 독립 비트맵	- BMP 파일 형태로 저장 - 색상 테이블, 해상도 등 모든 정보 포함 - 어떤 장치에서도 동일하게 출력 가능	파일 저장, 이미지 교환, 호환성 높은 출력

- DDB는 DIB에 비해 간단하며 DC에 바로 선택될 수 있는 비트맵
 - 프로그램 내부에서만 사용되는 비트맵의 경우에 많이 사용한다.
 - 장치에 의존적이기 때문에 원래 만들어진 장치 이외에서 출력할 경우 원래대로 출력되지 않을 수 있다.
 - 외부 비트맵파일(.bmp)을 프로그램에 불러와 그래픽 작업을 수행하거나 다양한 영상처리 효과를 주는 프로그램을 만드는 경우에는 장치에 독립적이고 훨씬 다양한 기능을 가지고 있는 DIB를 더 많이 사용한다

비트맵

- 비트맵 구조체 (DDB 비트맵)

```
typedef struct tagBITMAP {  
    LONG bmType;           // 비트맵 타입: 0  
    LONG bmWidth;          // 비트맵 가로 픽셀수  
    LONG bmHeight;         // 비트맵의 세로 픽셀수  
    LONG bmWidthBytes;     // 한 줄의 바이트 수  
    WORD bmPlanes;         // 색상 평면의 숫자  
    WORD bmBitsPixel;      // 각 픽셀당 색상을 위한 비트수 (예, 24비트, 32비트)  
    LPVOID bmBits;         // 실제 픽셀 데이터를 가르키는 포인터  
} BITMAP, *PBITMAP;
```

비트맵

- 비트맵 구조체 (DIB 비트맵)



<픽셀 당 1, 2, 4, 8비트 비트맵>



<픽셀 당 16, 24, 32 비트 비트맵>

비트맵

- 비트맵 구조체 (DIB 비트맵)

BITMAPFILEHEADER (비트맵 파일에 대한 정보)	<pre>typedef struct tag BITMAPFILEHEADER { WORD bfType; // 비트맵 파일 확인 (BM 타입) DWORD bfSize; // 비트맵 파일 크기 WORD bfReserved1; // 0 WORD bfReserved2; // 0 DWORD bfOffBits; // 실제 비트맵 데이터 값과 헤더의 오프셋 값 } BITMAPFILEHEADER, *PBITMAPFILEHEADER;</pre>
BITMAPINFOHEADER (비트맵 자체에 대한 정보)	<pre>typedef struct tag BITMAPINFOHEADER{ DWORD biSize; // BITMAPINFOHEADER 구조체 크기 LONG biWidth; // 비트맵의 가로 픽셀 수 LONG biHeight; // 비트맵의 세로 픽셀 수 WORD biPlanes; // 장치에 있는 색상 면의 개수 (반드시 1) WORD biBitCount; // 한 픽셀을 표현할 수 있는 비트의 수 (1, 4, 8, 16, 24, 32) DWORD biCompression; // 압축 상태 지정 (BI_RGB: 압축되지 않은 비트맵 DWORD biSizeImage; // 실제 이미지의 바이트 크기 (비 압축 시 0) LONG biXPelsPerMeter; // 미터당 가로 픽셀 수 LONG biYPelsPerMeter; // 미터당 세로 픽셀 수 DWORD biClrUsed; // 색상테이블의 색상 중 실제 비트맵에서 사용되는 // 색상수 (0 : 모든 색상을 사용 // 비트맵을 출력하는데 필수적인 색상인덱스수 // (0 : 모든 색상이 사용되어야 함) DWORD biClrImportant; } BITMAPINFOHEADER, *PBITMAPINFOHEADER;</pre>
RGBQUAD 배열 (색상 테이블)	<pre>typedef struct tag RGBQUAD { BYTE rgbBlue; // 파란색 BYTE rgbGreen; // 초록색 BYTE rgbRed; // 빨간색 BYTE rgbReserved; // 예약된 값: 0 } RGBQUAD;</pre>
color/index 배열 (픽셀 데이터)	24비트 비트맵 파일이 픽셀인 경우 BGRBGRBGR... (B: blue, G: green, R: red가 각각 1바이트씩, BGR이 한 픽셀)

비트맵

• 비트맵 읽기

- 비트맵 이미지는 **LoadBitmap()** 함수 또는 **LoadImage()** 함수로 읽는다.
 - LoadBitmap 함수:
 - 리소스로 불러온 이미지 파일만 읽기가 가능하다.
 - LoadImage 함수:
 - 리소스로 불러온 이미지 파일 또는 이미지 파일로도 읽기가 가능하다.
- 비트맵 이미지를 사용하기 위해서
 - 이미지 편집기에서 직접 이미지 만들어 활용하기
 - 이미 만들어져 있는 이미지를 다운로드 받아서 활용하기

BITMAPFILEHEADER

Little-Endian															
bfType				BfSize				bfReserved1				bfReserved2			
00000000h:				42	4D	68	00	00	00	00	00	00	00	00	28 00
00000010h:				00	00	04	00	00	00	04	00	00	00	01 00	18 00 00 00
00000020h:				00	00	32	00	00	00	12	0B	00	00	12 0B	00 00 00 00
00000030h:				00	00	00	00	00	00	FF	00	00	FF	00 00	50 50 50 50
00000040h:				50	50	FF	00	00	FF	00	00	50	50	50 50	50 50 00 00
00000050h:				FF	00	00	FF	00	FF	00	00	FF	00	00 00	FF 00 00 FF
00000060h:				00	FF	00	00	FF	00	00	00	00	00	00 00	

BITMAP FILE HEADER

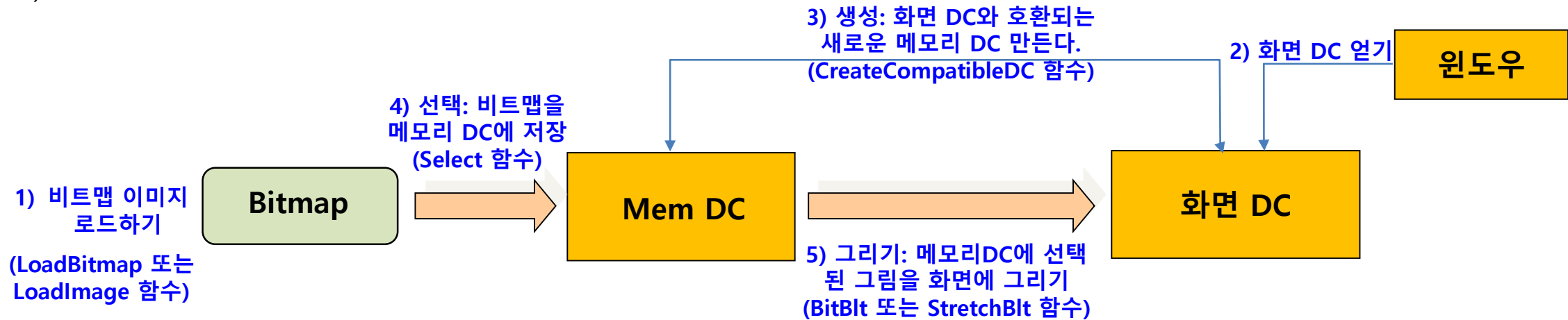
BITMAPINFOHEADER

biWidth				biHeight				biPlanes				biBitcount			
biSizeImage				biXPelsPerMeter				biYpelsPerMeter				biCompression			
biClrImportant				biXPelsPerMeter				biYpelsPerMeter				biClrUsed			
00000000h:				42	4D	68	00	00	00	00	00	36	00	00	28 00
00000010h:				00	00	04	00	00	00	04	00	01	00	18 00	00 00
00000020h:				00	00	32	00	00	00	12	0B	12	0B	00 00	00 00
00000030h:				00	00	00	00	00	00	FF	00	FF	00	50 50	50 50
00000040h:				50	50	FF	00	00	FF	00	00	50	50	50 50	50 00 00
00000050h:				FF	00	00	FF	00	FF	00	00	FF	00	00 00	FF 00 00 FF
00000060h:				00	FF	00	00	FF	00	00	00	00	00	00 00	<- Create Photoshop !!

비트맵 화면에 출력하기

• 비트맵 화면에 출력하기

- 1) 비트맵 로드하기
- 2) 화면 디바이스 컨텍스트 얻기
- 3) 메모리 디바이스 컨텍스트 만들기
- 4) 비트맵 사용하기 위해 선택하기
- 5) 비트맵 화면에 출력하기

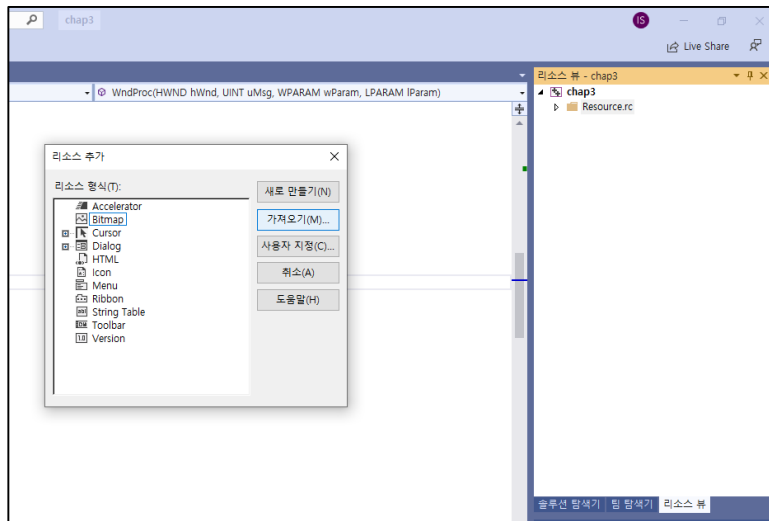


1) 비트맵 로드하기: 리소스로 사용하기

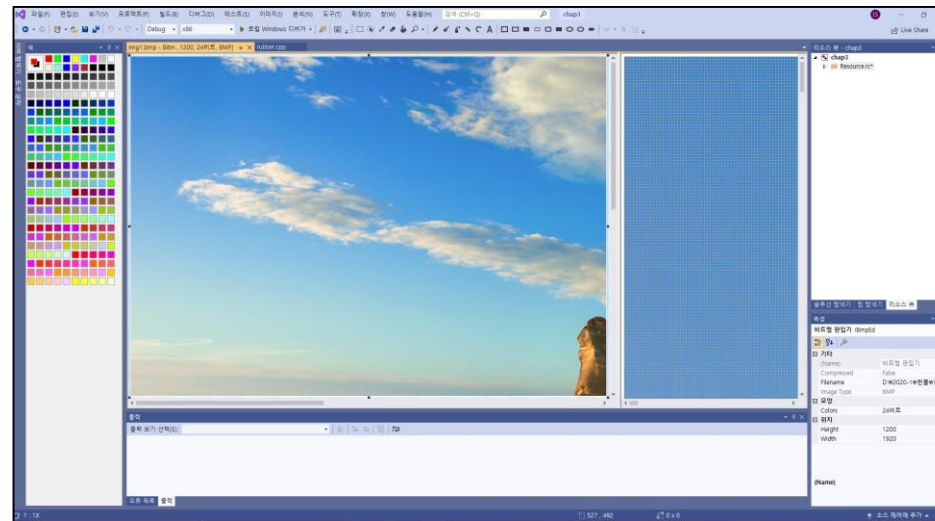
- 리소스 파일 작성
 - 방법: 소스 파일 작성과 유사
 - "C++ Source" 대신에 **Resource Script** 선택
 - 리소스 파일 이름 명시
- 비트맵을 리소스로 사용하려면
 - 새로운 비트맵 리소스 만들기
 - 새로 만들기: 직접 만들기 → 리소스 편집기에서 이미지 만들기
 - 불러오기: 이미 만들어진 이미지 활용하기 → 기존의 비트맵 이미지를 읽어서 사용하기

비트맵 로드하기: 리소스로 사용하기

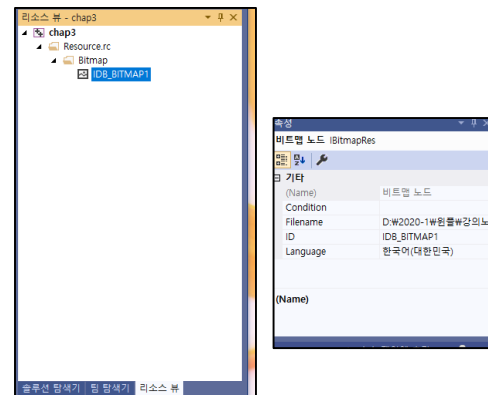
- Visual Studio 2022 환경
 - 리소스에서 비트맵 추가
 - 비트맵 편집창



비트맵 불러오기



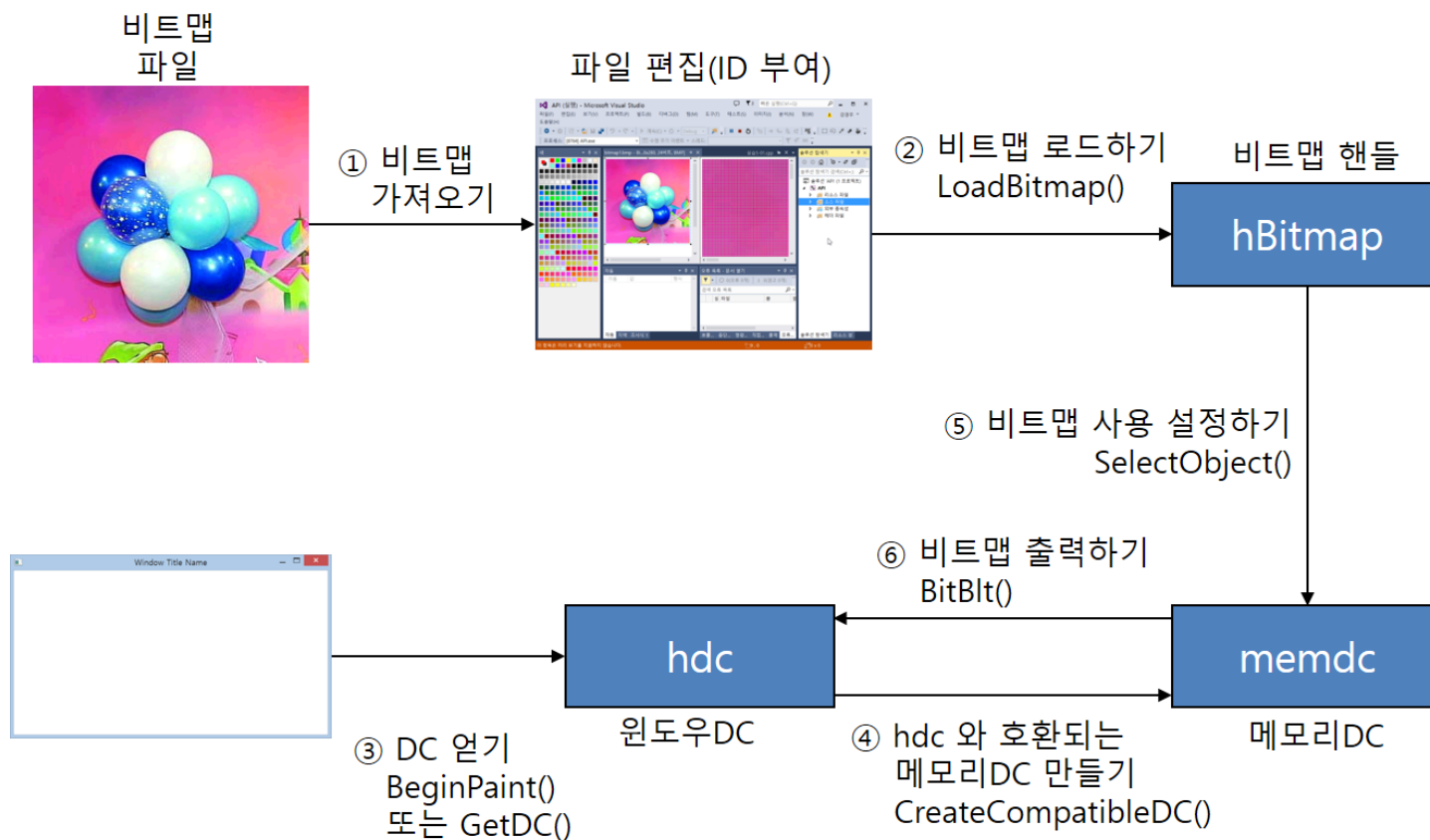
불러온 비트맵 편집창



리소스뷰와 비트맵 속성창

비트맵 로드하기: 리소스로 사용하기

- 비트맵 이미지를 출력하기



비트맵 로드하기: LoadBitmap () 함수

- 비트맵 읽기

- 비트맵을 읽어올 때: **LoadBitmap** 함수를 사용
 - 리소스로 불러온 이미지 파일만 읽기가 가능하다. 리소스 파일에 추가된 이미지는 빌드하면 실행 파일에 포함된다.
 - 이 함수로 읽은 비트맵은 DDB로 변경된다.

HBITMAP LoadBitmap (HINSTANCE hInstance, LPCTSTR lpBitmapName);

- 리소스를 사용하여 비트맵 로드
 - HINSTANCE hInstance: 어플리케이션 인스턴스 핸들
 - LPCTSTR lpBitmapName: 비트맵 리소스 이름

- 비트맵 읽기 예)

- LoadBitmap 함수 사용: 리소스로 사용

```
HBITMAP hBitmap;
```

```
hBitmap = LoadBitmap (hInstance, MAKEINTRESOURCE(IDB_BITMAP1));    //--- IDB_BITMAP1: 리소스로 저장된 이미지
```

2) 화면 디바이스 컨텍스트 얻기

- 디바이스 컨텍스트 얻기
 - 기존의 방식으로 DC 얻기
HDC hDC = GetDC(hWnd);
또는
HDC hDC = BeginPaint(hWnd, &ps);

3) 메모리 디바이스 컨텍스트 만들기

- 메모리 디바이스 컨텍스트 (메모리 DC)
 - 화면 DC와 동일한 특성을 가지며 그 내부에 출력 표면을 가진 메모리 영역
 - 화면 DC에서 사용할 수 있는 모든 출력을 메모리 DC에서 할 수 있다.
 - 메모리 DC에 먼저 그림을 그린 후 사용자 눈에 그려지는 과정은 보여주지 않고 메모리 DC에서 작업을 완료한 후 그 결과만 화면으로 고속 복사한다.
 - 비트맵도 일종의 GDI 오브젝트이지만 화면 DC에서는 선택할 수 없으며 메모리 DC만이 비트맵을 선택할 수 있어서 메모리 DC에서 먼저 비트맵을 읽어온 후 화면 DC로 복사한다.
- 메모리 DC를 만들 때: CreateCompatibleDC 함수 사용

HDC CreateCompatibleDC (HDC hDC);

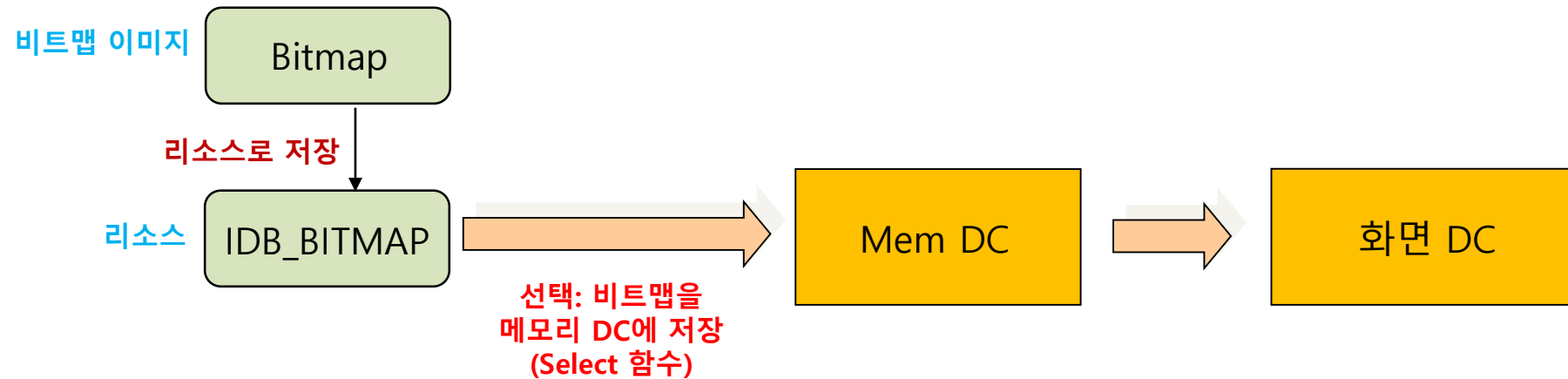
- 주어진 DC와 호환되는 메모리 DC를 생성해서 리턴한다.
- HDC hDC: 주어진 DC

4) 비트맵 선택하기

- 비트맵 선택
 - 메모리 DC를 만든 후에는 읽어온 비트맵을 메모리 DC에 선택해 준다.
 - 선택하는 방법: **SelectObject** 함수를 사용

HGDIOBJ SelectObject (HDC hDC, HGDIOBJ hgdiobj);

- hDC: DC 핸들값
- hgdiobj: GDI의 객체
- 리턴 값은 원래의 오브젝트 값



비트맵: 리소스로 사용하기

- 비트맵 이미지 읽어 화면에 출력하기

```
LRESULT CALLBACK WndProc(HWND hWnd, UINT iMsg, WPARAM wParam, LPARAM lParam)
{
    HDC hDC, hMemDC;
    HBITMAP hBitmap;

    switch (iMsg)
    {
        case WM_PAINT:
            hDC = BeginPaint(hWnd, &ps);

            hBitmap = LoadBitmap (hInstance, MAKEINTRESOURCE(IDB_BITMAP1));    //--- 리소스로 읽은 비트맵을 로드하기

            hMemDC = CreateCompatibleDC (hDC);                                //--- 1. 주어진 DC와 호환되는 DC를 생성
            SelectObject (hMemDC, hBitmap);                                   //--- 2. 새로 만든 DC에 그림을 선택한다

            BitBlt (hDC, 0, 0, 320, 320, hMemDC, 0, 0, SRCCOPY);              //--- 3. DC간 블록 전송을 수행한다
            ...

    }
}
```

5) 비트맵 출력하기: BitBlt()

- **BitBlt 함수**

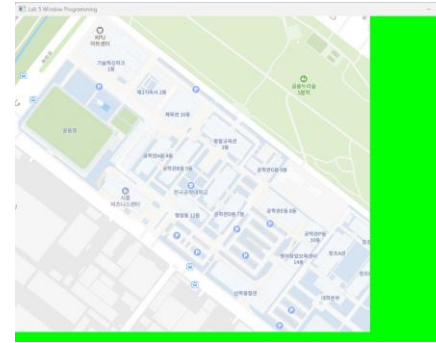
- DC간의 영역끼리 고속 복사 수행
- 메모리DC에 그려져 있는 비트맵을 화면 DC로 복사하여 비트맵을 화면에 출력
- 1:1로 영역 복사

BOOL BitBlt (**HDC hDC**, **int nXD**, **int nYD**, **int nW**, **int nH**,
HDC hMemDC, **int nXS**, **int nYS**, **DWORD dwRop**);

- DC 간의 영역 고속 복사
- 메모리 DC의 표면에 그려져 있는 비트맵을 화면 DC로 복사하여 비트맵을 화면에 출력
 - hDC: 복사 대상 DC
 - nXD, nYD: 복사 대상의 x, y 좌표 값
 - nW, nH: 복사 대상의 폭과 높이
 - hMemDC: 복사 소스 DC
 - nXS, nYS: 복사 소스의 좌표
 - dwRop: 래스터 연산 방법
 - BLACKNESS : 검정색으로 칠한다.
 - DSTINVERT: 대상의 색상을 반전시킨다.
 - MERGECOPY: 이미지 색상과 현재 선택된 브러시를 AND 연산시킨다.
 - MERGEPAINT: 반전된 이미지와 화면의 색을 OR 연산시킨다.
 - NOTSRCCOPY: 소스값을 반전시켜 칠한다.
 - SRCPAINT: 소스와 대상의 OR연산 값으로 칠한다.
 - **SRCCOPY**: 소스값을 그대로 칠한다.
 - SRCAND: 소스와 대상의 AND연산 값으로 칠한다.
 - WHITENESS: 흰색으로 칠한다.

5) 비트맵 출력하기: BitBlt()

- BitBlt 함수에서 다양한 래스터 연산 적용 결과



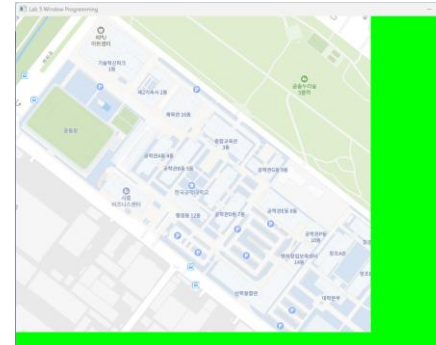
SRCCOPY 연산
(윈도우 배경색을 (0, 255, 0)으로
설정하였음)



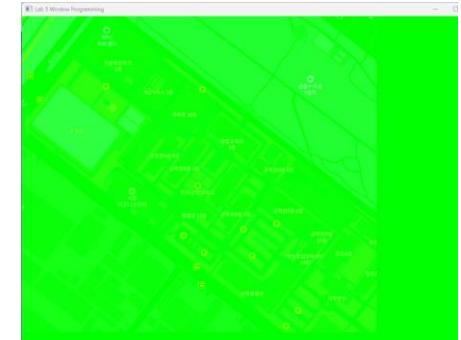
BLACKNESS



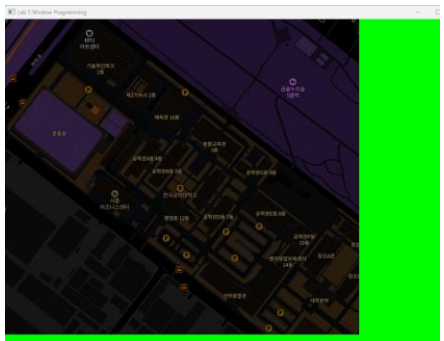
DSTINVERT



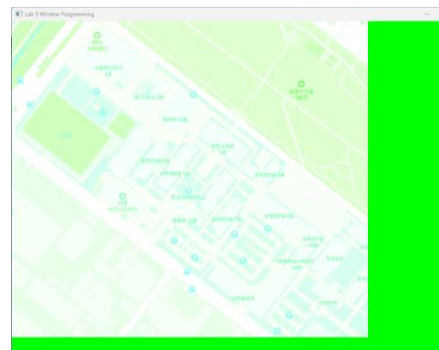
MERGECOPY



MERGEPAINT



NOTSRCCOPY



SRCPAINT

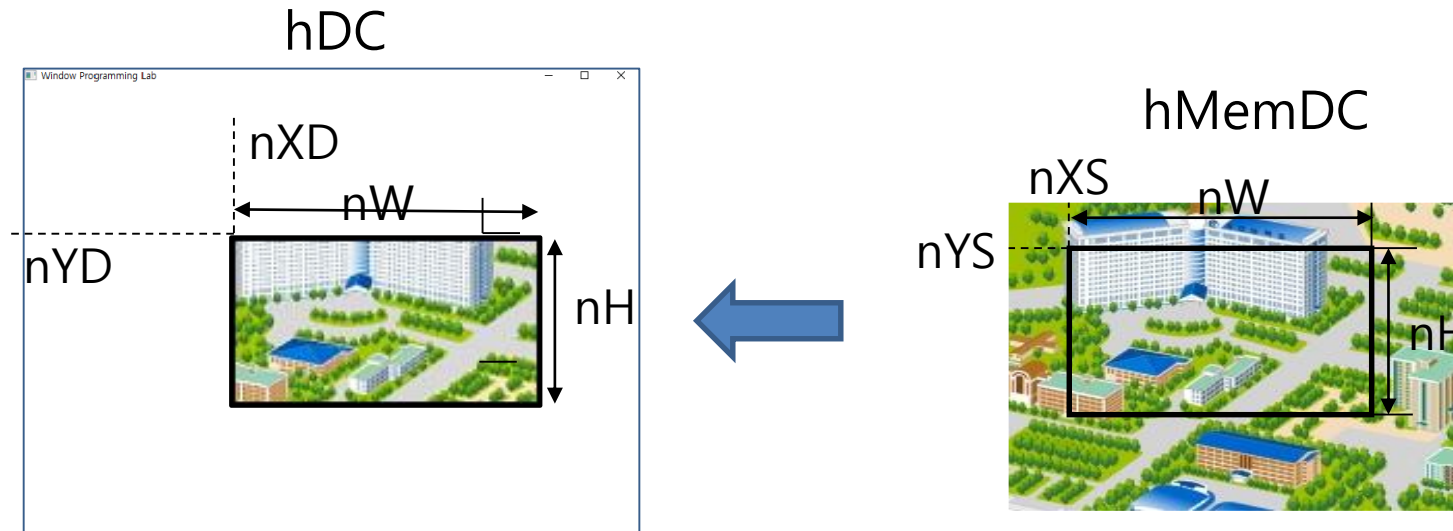


SRCAND



WHITENESS

BitBlt() → 1 : 1 영역 복사



```
BitBlt ( hDC, nXD, nYD, nW, nH, hMemDC, nXS, nYS, SRCCOPY);
```

• 비트맵 출력 후, 메모리 DC와 비트맵 해제

BOOL DeleteDC (HDC hDC);

- 생성한 메모리 DC를 제거한다.
 - HDC hDC: 제거 할 DC

BOOL DeleteObject (GDIOBJ hObject);

- 생성한 비트맵 객체를 제거한다.
 - GDIOBJ hObject: 제거할 객체

```
LRESULT CALLBACK WndProc (HWND hWnd, UINT iMessage, WPARAM wParam, LPARAM lParam)
{
    HDC hDC, hMemDC ;
    PAINTSTRUCT ps ;
    static HBITMAP hBitmap;

    switch (iMessage) {
        case WM_CREATE:
            hBitmap = (HBITMAP) LoadBitmap (g_hInst, MAKEINTRESOURCE (IDB_BITMAP1));    //---1) 비트맵 로드하기
            break;

        case WM_PAINT:
            hDC = BeginPaint (hWnd, &ps);
            hMemDC = CreateCompatibleDC (hDC);
            SelectObject (hMemDC, hBitmap);
            BitBlt (hDC, 0, 0, 320, 240, hMemDC, 0, 0, SRCCOPY);
            //--- 2) 화면 DC 얻어오기
            //--- 3) 메모리 DC 만들기
            //--- 4) 비트맵 선택하기
            //--- 5) 비트맵 출력하기
            //--- hMemDC 에 있는 그림에서 (0, 0) 위치에 (320, 240) 크기로 그리기
            //--- SRCCOPY : 바탕색과 관계없이 소스값을 그대로 그리기

            DeleteDC (hMemDC);
            EndPaint (hWnd, &ps);
            break;

        case WM_DESTROY:
            DeleteObject (hBitmap);
            PostQuitMessage (0);
            break;
    }
    return DefWindowProc (hWnd, iMessage, wParam, lParam);
}
```

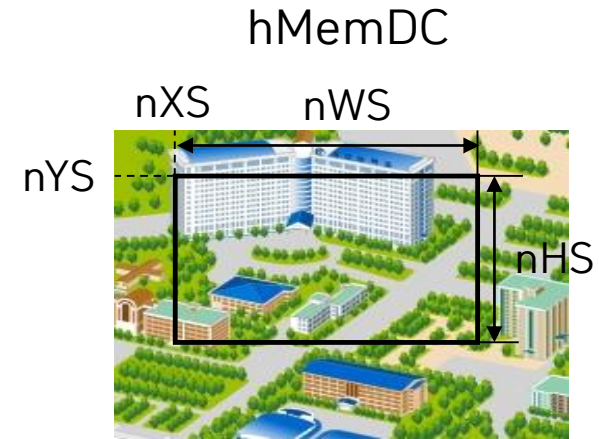
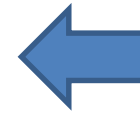
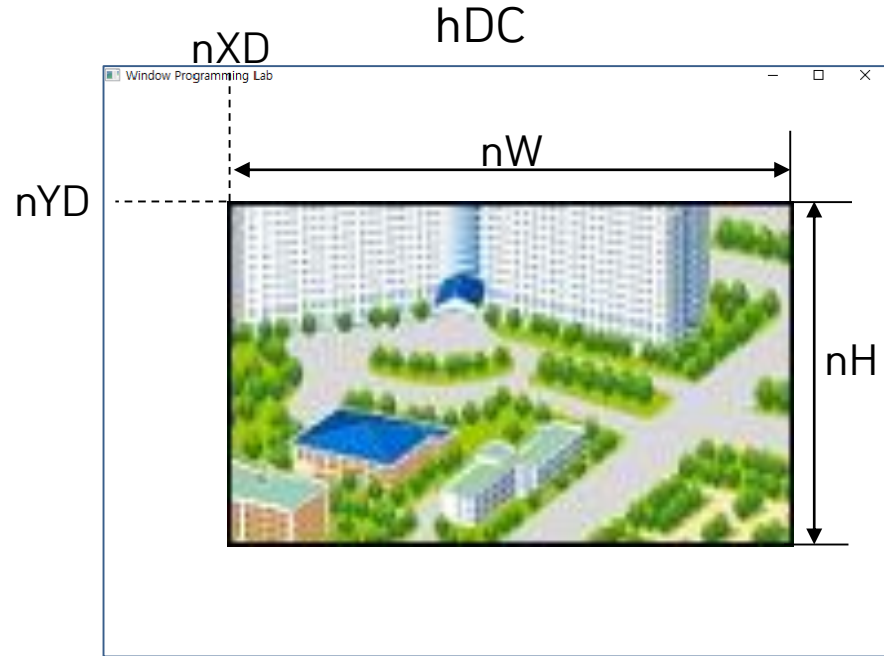
5) 비트맵 출력하기: StretchBlt()

- **StretchBlt 함수**
 - BitBlt 함수와 유사하게 DC간에 비트맵을 전송하여 복사
 - 복사원의 크기와 높이를 따로 지정하여 확대 및 축소 복사할 수 있다.

```
BOOL StretchBlt ( HDC hDC, int nXD, int nYD, int nW, int nH,  
                  HDC hMemDC, int nXS, int nYS, int nWS, int nHS,    DWORD dwRop );
```

- DC간의 이미지 확대 또는 축소하여 복사
 - hDC: 복사대상 DC
 - nXD, nYD: 복사대상 DC x, y 좌표값
 - nW, nH: 복사대상 DC의 폭과 높이
- hMemDC: 복사소스 DC
- nXS, nYS: 복사소스 DC의 x, y 좌표값
- nWS, nHS: 복사소스 DC의 폭과 높이
- dwRop: 래스터 연산 방법
 - BLACKNESS : 검정색으로 칠한다.
 - DSTINVERT: 대상의 색상을 반전시킨다.
 - MERGECOPY: 이미지 색상과 현재 선택된 브러시를 AND 연산시킨다.
 - MERGEPAINT: 반전된 이미지와 화면의 색을 OR 연산시킨다.
 - NOTSRCCOPY: 소스값을 반전시켜 칠한다.
 - SRCPAINT: 소스와 대상의 OR연산 값으로 칠한다.
 - **SRCCOPY: 소스값을 그대로 칠한다.**
 - SRCAND: 소스와 대상의 AND연산 값으로 칠한다.
 - WHITENESS: 흰색으로 칠한다.

StretchBlt() → 확대, 축소 영역 복사



StretchBlt (hDC , nXD , nYD , nW , nH ,

$hMemDC$, nXS , nYS , nWS , nHS , $SRCCOPY$);

```
LRESULT CALLBACK WndProc(HWND hWnd, UINT iMessage, WPARAM wParam, LPARAM lParam)
```

```
{
    HDC hDC, hMemDC ;
    PAINTSTRUCT ps ;
    static HBITMAP hBitmap;

    switch (iMsg) {
        case WM_CREATE:
            hBitmap = (HBITMAP) LoadBitmap (hInstance, MAKEINTRESOURCE (IDB_BITMAP1));
            break;

        case WM_PAINT:
            hDC = BeginPaint(hWnd, &ps);
            hMemDC = CreateCompatibleDC (hDC);
            SelectObject (hMemDC, hBitmap);
            StretchBlt (hDC, 100, 0, 160, 120, hMemDC, 0, 0, 320, 240, SRCCOPY);
            //--- 메모리 DC에 있는 그림에서 (0, 0)위치에서 (320, 240) 크기의 그림을
            //--- 화면의 (100, 0)위치에 (160, 120) 크기로 이미지 색 그대로 그리기

            DeleteDC (hMemDC);
            EndPaint (hWnd, &ps);
            break;
        case WM_DESTROY:
            DeleteObject (hBitmap);
            PostQuitMessage (0);
            break;
    }
    return DefWindowProc (hWnd, iMessage, wParam, lParam);
}
```

1)-2 비트맵 로드하기: LoadImage () 함수 (리소스 대신 이미지로 직접 사용하기)

- 비트맵 로드하기
 - 비트맵을 읽어올 때: LoadImage 함수를 사용
 - 리소스로 불러온 이미지 파일 또는 이미지 파일 자체로도 읽기가 가능하다.

HANDLE LoadImage (HINSTANCE hInstance, LPCTSTR lpBitmapName, UINT uType, int cxDesired, int cyDesired, UINT fuLoad);

- 비트맵, 아이콘, 커서를 로드 (LoadIcon, LoadCursor, LoadBitmap 함수를 통합)
 - HINSTANCE hInstance: 어플리케이션 인스턴스 핸들
 - LPCTSTR lpBitmapName: 비트맵 리소스 이름
 - 비트맵을 리소스 또는 파일 이름으로 사용할 수 있음.
 - 파일이름인 경우 fuLoad에 LR_LOADFROMFILE 플래그를 설정
 - UINT uType: 불러올 이미지의 종류
 - **IMAGE_BITMAP**: 비트맵 불러오기
 - IMAGE_ICON: 아이콘 불러오기
 - IMAGE_CURSOR: 커서 불러오기
 - int cxDesired, cyDesired: 아이콘이나 커서를 읽을 경우 아이콘이나 커서의 너비, 높이
 - UINT fuLoad: 플래그 설정
 - LR_DEFAULTCOLOR: 기본 플래그로 이미지를 흑백으로 불러오지 않도록 함
 - **LR_LOADFROMFILE**: lpBitmapName을 리소스 대신 파일이름을 사용해 불러옴
 - LR_DEFAULTSIZE: cxDesired, cyDesired 인자가 0인 경우 시스템 지정값 사용
 - **LR_CREATEDIBSECTION**: uType 인수에서 IMAGE_BITMAP을 사용한 경우 호환 비트맵이 아닌 DIB 섹션 비트맵으로 불러옴

비트맵 로드하기: LoadImage () 함수

```
LRESULT CALLBACK WndProc(HWND hWnd, UINT iMessage, WPARAM wParam, LPARAM lParam)
{
    HDC hDC, hMemDC ;
    PAINTSTRUCT ps ;
    static HBITMAP hBitmap;

    switch (iMessage) {
        case WM_CREATE:
            //-- 리소스 대신 비트맵 직접 로드
            hBitmap = (HBITMAP) LoadImage (g_hInst, TEXT("image.bmp"), IMAGE_BITMAP, 0, 0, LR_LOADFROMFILE | LR_CREATEDIBSECTION);
            break;

        case WM_PAINT:
            hDC = BeginPaint(hWnd, &ps);
            hMemDC = CreateCompatibleDC (hDC);
            SelectObject (hMemDC, hBitmap);
            StretchBlt (hDC, 100, 0, 160, 120, hMemDC, 0, 0, 320, 240, SRCCOPY);
            DeleteDC (hMemDC);
            EndPaint (hWnd, &ps);
            break;

        case WM_DESTROY:
            DeleteObject (hBitmap);
            PostQuitMessage (0);
            break;
    }
    return DefWindowProc (hWnd, iMessage, wParam, lParam);
}
```

GetObject (): 그림 크기 알아내기

- 그림 크기 알아내기

```
int GetObject ( HGDIOBJ hgdioobj, int cbBuffer, LPVOID lpvObject);
```

- 객체의 정보 알아오기
 - HGDIOBJ hgdioobj: GDI 오브젝트 핸들
 - int cbBuffer: 오브젝트 버퍼의 크기에 관한 정보
 - LPVOID lpvObject: 오브젝트 정보 버퍼를 가리키는 포인터

- 사용 예)

```
BITMAP bmp;                //--- DDB 포맷의 비트맵 이미지 타입
int mWidth, mHeight;
```

```
GetObject (hBitmap, sizeof(BITMAP), &bmp);
mWidth = bmp.bmWidth;
mHeight = bmp.bmHeight;
```

```
typedef struct tagBITMAP {
    LONG bmType;           // 비트맵 타입: 0
    LONG bmWidth;          // 비트맵의 넓이 (픽셀 단위)
    LONG bmHeight;         // 비트맵의 높이 (픽셀 단위)
    LONG bmWidthBytes;     // 각 스캔 라인의 바이트 수
    WORD bmPlanes;         // 색상 판의 숫자
    WORD bmBitsPixel;      // 각 픽셀당 색상을 위한 비트수
    LPVOID bmBits;         // 비트맵을 가리키는 포인터
} BITMAP, *PBITMAP;
```

비트맵 읽어 화면에 출력하는 예

- 크기를 모르는 비트맵 이미지를 리소스로 읽어 화면에 맞게 출력하기

```
LRESULT CALLBACK WndProc(HWND hwnd,UINT iMsg,WPARAM wParam,LPARAM lParam)
```

```
{
static BITMAP bmp;
static HBITMAP hBitmap;
static RECT rect;
HDC hDC, hMemDC;
```

```
switch (iMsg) {
case WM_CREATE:
    hBitmap = (HBITMAP)LoadBitmap(g_hInst, MAKEINTRESOURCE(IDB_BITMAP1));
    GetClientRect (hwnd, &rect);
    GetObject (hBitmap, sizeof(BITMAP), &bmp);
    break;
case WM_PAINT:
    hDC = BeginPaint(hwnd, &ps);
    hMemDC = CreateCompatibleDC (hDC);
    (HBITMAP) SelectObject (hMemDC,hBitmap);
    StretchBlt (hDC, 0, 0, rect.right, rect.bottom, hMemDC, 0, 0, bmp.bmWidth, bmp.bmHeight, SRCCOPY);
    DeleteDC (hMemDC);
    EndPaint (hwnd, &ps);
    break;
case WM_DESTROY:
    PostQuitMessage(0);
    break;
}
return(DefWindowProc(hwnd,iMsg,wParam,lParam));
}
```

```
//--- 리소스로 비트맵 읽어오기
//--- 윈도우 크기 읽어오기
//--- 비트맵 정보 읽어오기 (크기 포함)
```

```
//--- 메모리 DC 만들기
//--- 비트맵과 메모리 DC 연결하기
//--- 윈도우 크기에 맞기 비트맵 그려주기
```

